

Artificial Intelligence, Blockchain & Criptovalute nello Sviluppo Software

Manuale Intensivo d'Esame — Piano 3 Giorni

■ Università di Bologna | ■■■ Prof. Giancarlo Succi | ■ Primavera 2026 | ■ LM Informatica

■■ ANALISI PRELIMINARE — Cosa è DAVVERO importante per l'esame

Natura dell'esame: Orale onnicomprensivo. Il docente Succi collega psicologia, ingegneria del software, AI e sistemi socio-tecnici. Rispondere come se fosse un esame di **teoria dei sistemi umani applicata al software**, non di programmazione.

■ Argomenti certamente all'esame:

- Problemi **Tame vs Wicked** — il software come problema Wicked
- Sistemi **PDS / DCS / StS / SR** (Mensch) — regimi dinamici
- Modello **Impulsivo/Riflessivo** (Strack & Deutsch)
- **EEG**, onde cerebrali, ERD/ERS applicati ai programmatori
- **AI = Modeling + Emulating** la mente umana
- **Blockchain** come sistema decentralizzato a retroazione negativa
- **Manifesto Agile / XP / 7 Sprechi Lean**
- **Matrice di Harris** per il controllo del processo
- **Bateson**: informazione come "differenza che produce differenza"

■ Gap nel riassunto Gemini (aree da approfondire):

- Matematica dei sistemi caotici (equazioni differenziali, sensibilità)

- 5 assiomi di Watzlawick — comunicazione
- Schismogenesi di Bateson
- Pipeline MNE-Python per EEG + formula IAF

■ Ingegneria del Software — Fondamenta Concettuali

Capitolo 1 — La Natura del Software e la sua Economia

■ KEYWORDS

costo marginale zero · copyright · licenza · segmentazione · bundling · economia di rete · diritti morali · diritti economici

■ CONCETTI CHIAVE

Il software è il risultato di un **atto creativo della mente umana** ed è protetto dal diritto d'autore (dura 50 anni dalla morte dell'autore, vs 20 anni dei brevetti). Il software **non viene venduto, viene concesso in licenza**.

Funzione di costo a "L": il costo di sviluppo è altissimo (costo fisso), ma il **costo marginale di ogni copia è quasi zero**. Questo abilita strategie aggressive di segmentazione.

Diritti morali (non alienabili): paternità dell'opera. **Diritti economici** (alienabili): possono essere ceduti al datore di lavoro.

Strategia	Descrizione	Esempio
Segmentazione	Prezzi diversi per segmenti diversi	Office Professional vs Educational
Bundling	Pacchetto di prodotti per promuovere nuovi	Suite Office completa
Freemium	Base gratuita + premium a pagamento	Spotify, LinkedIn

■ ERRORI COMUNI

- Non confondere "venduto" con "licenziato": il software non si vende mai, si licenzia.
- Non dire che le economie di scala funzionano come nel manifatturiero: la replica è gratuita, ma la rigidità Fordista fallisce di fronte alla variabilità.

Capitolo 2 — La Software Crisis e le 4 Difficoltà di Brooks

■ KEYWORDS

software crisis · Dijkstra 1972 · Fred Brooks · No Silver Bullet · complessità · conformità · modificabilità · invisibilità · Ariane 5

■ CONCETTI CHIAVE

Software Crisis (Dijkstra, 1972): la potenza hardware cresce più velocemente della capacità umana di gestire la programmazione.

Le 4 difficoltà strutturali di Fred Brooks:

#	Difficoltà	Spiegazione
1	Complessità	Il software ha più stati possibili di qualsiasi sistema fisico.
2	Conformità	Deve adattarsi a leggi, istituzioni e interfacce preesistenti (spesso irrazionali).
3	Modificabilità	Il successo genera richieste di cambiamento continue.
4	Invisibilità	Essendo immateriale, è difficile da visualizzare e comunicare.

■ CASO PRATICO — Ariane 5

■■ **Caso Ariane 5 (1996):** Modulo software di Ariane 4 riutilizzato in Ariane 5 senza verificare la compatibilità di contesto. Un overflow (64-bit in 16-bit) distrusse il razzo. **Danni: ~8 miliardi di dollari.** Lezione: il software non è un componente fisico intercambiabile.

■ DOMANDE POSSIBILI

- "Quali sono le 4 difficoltà di Brooks? Collegale ad un caso reale."
- "Cos'è la software crisis? Perché è ancora rilevante?"
- "Spieghi il caso Ariane 5 e cosa ci insegna sul riutilizzo del software."

Capitolo 3 — Teoria dei Sistemi e Regimi di Mensch

■ KEYWORDS

PDS Pseudo-Deterministico · **DCS** Caotico Deterministico · **StS** Stocastico · **SR** Randomicità Stocastica · **effetto farfalla** · **illusione del tacchino**

Regime	Sigla	Caratteristica	Nel Software
Pseudo-Deterministico	PDS	Stabile "per ingegneria", nasconde fragilità (illusione del tacchino)	Waterfall sembra funzionare... fino al crollo
Caotico Deterministico	DCS	Equazioni precise ma sensibilità estrema alle condizioni iniziali	Bug impossibili da riprodurre; comportamenti erratici
Stocastico	StS	Incertezza ma previsione statistica possibile	Sprint Agile: si stima la probabilità
Randomicità Stocastica	SR	Caos puro, solo contenimento danni	Cyberattack inaspettato, crash totale

$$f''(x) = a \cdot f'(x) + b \cdot f(x)$$

$a = -0.1, b = -1 \rightarrow$ oscillazione smorzata (stabile)

$a = +0.1, b = -1 \rightarrow$ oscillazione divergente (instabile)

$\rightarrow$ Una piccola variazione di 'a' cambia completamente il comportamento!

■ COLLEGAMENTO FONDAMENTALE

I regimi di Mensch spiegano *perché* il Taylorismo (PDS) fallisce nel software (che è DCS/StS) e perché serve l'Agile. Blockchain è un sistema che cerca di essere PDS tramite il consensus protocol.

Capitolo 4 — Problemi Tame e Wicked

■ KEYWORDS

Tame Problem · Wicked Problem · Rittel & Webber · soluzione accettabile · formulazione iterativa

Caratteristica	Tame (Mansueto)	Wicked (Perverso)
Formulazione	Ben definita, stabile	Non definitiva; cambia mentre si risolve
Soluzione	"Giusta" o "sbagliata"	"Accettabile" o "non accettabile"
Classi	Appartiene a classi note	Unico, non ripetibile
Approccio	Waterfall, Taylorismo	Agile, iterativo, stakeholder continui

■■ ERRORI COMUNI

Non dire che Wicked = "difficile". Wicked significa strutturalmente non formulabile in modo definitivo. Un problema può essere facile ma Wicked. Il software è quasi sempre un problema Wicked.

Capitolo 5 — Meccanismi di Coordinamento e Matrice di Harris

■ KEYWORDS

Malone & Crowston · coordinamento sequenziale · risorse condivise · risultato comune · Taylorismo · Fordismo · Matrice di Harris · Waterfall · Lean · Agile

Tipo	Descrizione	Esempio SW	Pro/Con
Sequenziale	Deliverable passa da reparto a reparto	Waterfall classico	■ Semplice ■ Rigido
Risorse Condivise	Tutti pescano da una coda comune	Kanban, backlog	■ Flessibile ■ Richiede priorità
Risultato Comune	Team lavora insieme sullo stesso output	Pair Programming, mob prog.	■ Ottimale per Wicked ■ Difficile

■ MATRICE DI HARRIS

Matrice di Harris — Controllo del Processo ■ CRITICO

Tipo	Si misura	Quando si usa	Esempio
Outcome	Il risultato finale	Conosco relazione causa-effetto	KPI vendite, test superati
Behavior	I passaggi del processo	Conosco il processo (Taylorismo)	Ore di codice, commit/giorno
Clan/Group	Nessuna misurazione diretta	Processo Wicked, serve fiducia (Agile)	Daily scrum, retrospettiva

■■ ERRORI COMUNI

■■ Questa matrice è esplicitamente citata nell'appendice "Focus del Docente" del materiale — **altissima probabilità all'esame.**

■ Modelli Cognitivi nella Produzione Software

Capitolo 6 — Bateson e la Teoria della Percezione

■ KEYWORDS

Gregory Bateson · informazione · differenza · abduzione · double bind · schismogenesi · Watzlawick · 5 assiomi comunicazione

■ CONCETTI CHIAVE

Informazione (Bateson): "Una differenza che produce una differenza." Non tutti i dati sono informazione: un dato diventa informazione solo quando produce un cambiamento nel sistema che lo riceve.

Abduzione: Ragionamento per pattern — trovare il pattern migliore per spiegare le osservazioni. A differenza di induzione (particolare→generale) e deduzione (generale→particolare). È il modo in cui funzionano gli LLM.

Double Bind: Due messaggi contraddittori a livelli logici diversi creano un loop impossibile. Es: "Sii spontaneo!" Nel software: "Sviluppa velocemente E senza bug."

Schismogenesi (Bateson):

- **Complementare:** I comportamenti si rinforzano asimmetricamente (dominante↑→sottomesso↑→dominante↑...). Esempio: gatto e cane.
- **Simmetrica:** I comportamenti si rinforzano simmetricamente. Esempio: corsa agli armamenti; persone in fila allo stadio che si alzano progressivamente.

#	Assioma (Watzlawick)	Esempio nel SW
1	Non si può non comunicare	Il silenzio in code review comunica qualcosa
2	Contenuto + Relazione	Stesso messaggio con toni diversi cambia la relazione
3	Punteggiatura della sequenza	Bug: colpa del dev che ha codificato o del tester?
4	Digitale + Analogico	Stand-up: parole + linguaggio del corpo
5	Simmetrico vs Complementare	Team di pari vs junior con senior

Capitolo 7 — Modelli d'Uso della Conoscenza

■ KEYWORDS

naive scientist · cognitive miser · motivated tactician · reflective/impulsive model · Strack & Deutsch · System 1 / System 2 · attribution theory · heuristic

Modello	Metafora	Processo	Applicazione SW
---------	----------	----------	-----------------

Naive Scientist (anni '50-70)	Lo scienziato ingenuo	Raccoglie dati, forma ipotesi, testa razionalmente.	Dev che documenta tutto e segue best practice
Cognitive Miser (anni '80)	L'avaro cognitivo	Usa euristiche per minimizzare lo sforzo. "Abbastanza buono".	Copia-incolla Stack Overflow; primo pattern che funziona
Motivated Tactician (anni '90)	Il tattico motivato	Sceglie la strategia cognitiva in base al contesto.	Sa quando fare TDD rigoroso e quando "hackerare"
Reflective/Impulsive (2004)	Sistema 1 & 2	Due sistemi paralleli che a volte si scontrano.	Code review: impulso vs analisi critica

■ MODELLO IMPULSIVO/RIFLESSIVO

Il Modello Impulsivo/Riflessivo (Strack & Deutsch, 2004) ■ CRITICO

	Sistema Impulsivo	Sistema Riflessivo
Natura	Implicito, automatico	Esplicito, deliberato
Velocità	Veloce (millisecondi)	Lento (secondi/minuti)
Base	Memoria associativa (pattern)	Ragionamento logico
Effort	Nessuno sforzo	Alto sforzo cognitivo
Nel SW	Intuire un code smell; riconoscere un pattern	Analizzare complessità algoritmica; debuggare race condition

■ APPLICAZIONE PRATICA

Applicazione alle Code Review: Il sistema impulsivo domina sotto pressione ("sembra ok"). Per attivare il sistema riflessivo servono: checklist esplicite, tempo adeguato, separazione tra lettura e valutazione. Gli strumenti AI di code review replicano il sistema impulsivo, non quello riflessivo.

■ DOMANDE POSSIBILI

- "Spieghi il modello impulsivo/riflessivo e la sua applicazione alle software inspections."
- "Quali sono le differenze tra naive scientist, cognitive miser e motivated tactician?"
- "Come influisce il sistema impulsivo sulla qualità del software?"

Capitolo 8 — Mente Estesa, Cognizione Distribuita e Pensiero Sistemico

■ KEYWORDS

Clark & Chalmers · extended mind · Vygotsky · ZPD · scaffolding · Weinberg · homeostasis · etnografia cognitiva · Asch & Milgram · conformismo

■ CONCETTI CHIAVE

Mente Estesa (Clark & Chalmers)

L'ambiente e gli strumenti fanno fisicamente parte del processo cognitivo. Esempio: Otto (Alzheimer) usa un taccuino come memoria — funzionalmente equivalente alla memoria biologica.

Nel software: IDE, documentazione, Stack Overflow, Copilot sono estensioni della mente del programmatore. Un ambiente povero è una menomazione cognitiva reale.

Vygotsky e la ZPD (Zona di Sviluppo Prossimale)

Lo spazio tra ciò che so fare da solo e ciò che potrei fare con un mentore. Il **Pair Programming** si basa esattamente su questo principio.

Scaffolding: Il supporto temporaneo che un esperto fornisce, ritirandosi gradualmente man mano che il novizio cresce.

Pensiero Sistemico (Weinberg) + Rischio Conformismo

Homeostasis: I team resistono al cambiamento per mantenere l'equilibrio. Ogni nuova pratica (TDD, refactoring) trova resistenza sistemica.

Conformismo (Asch & Milgram): La pressione sociale può spingere sviluppatori a accettare soluzioni errate o a compiere atti non etici. La peer review non è automaticamente obiettiva.

Artificial Intelligence e Neuroscienze

Capitolo 9 — AI nello Sviluppo Software

KEYWORDS

AI = Modeling + Emulating · machine learning · data science · fuzzy models · NLP · reasoning · computational intelligence · LLM

Asse	Descrizione	Esempi
Modeling (Modellazione)	Usare l'AI per capire come funziona la mente umana. L'AI come strumento di ricerca cognitiva.	EEG per studiare i programmatori; analisi pattern di commit; predire il carico cognitivo
Emulating (Emulazione)	Algoritmi che replicano le azioni umane. L'AI come strumento di automazione.	ML, NLP, Copilot, test generation, code review automatica

Applicazione AI nel SW	Tipo AI	Esempio pratico
Predizione bug/qualità	ML (supervised)	Modelli che predicono bug dalle metriche di commit
Code completion	LLM	GitHub Copilot, Cursor, Claude Code
Smart contract analysis	ML + Static Analysis	Individuare vulnerabilità Solidity prima del deploy
Studio del programmatore	EEG + ML	Rilevare il carico cognitivo durante debugging
Test generation	LLM + Fuzzing	Generazione automatica di test case

COLLEGAMENTO FONDAMENTALE

AI = Emulating → riproduce le azioni del cognitive miser o del naive scientist? Quando usiamo Copilot, stiamo esternalizzando il sistema riflessivo o quello impulsivo? Questi collegamenti interdisciplinari sono tipici dello stile del docente.

Capitolo 10 — EEG e Segnali Cerebrali nei Programmatori

KEYWORDS

EEG · elettrodi 10-20 · onde Delta/Theta/Alpha/Beta/Mu · ERD · ERS · IAF · MNE-Python · notch filter · neuroni specchio

Strumento	Risoluzione Temporale	Costo	Portabilità
EEG ■	Alta (millisecondi)	Basso	Alta
fMRI	Bassa (secondi)	Altissimo	Nulla
PET	Bassissima	Altissimo	Nulla

Onda	Frequenza	Stato	Rilevanza SW
------	-----------	-------	--------------

Delta	1-5 Hz	Sonno profondo, inconscio	—
Theta	4-8 Hz	Carico cognitivo, memoria di lavoro	↑ durante problem solving difficile
Alpha	8-12 Hz	Rilassamento vigile. ↓ = sforzo	↓ durante debugging intenso (ERD)
Mu	7.5-12.5 Hz	Attività motoria, neuroni specchio	Pair programming (osservare codice)
Beta	12-25 Hz	Concentrazione attiva, pensiero analitico	↑ durante review, pianificazione

■ CONCETTI CHIAVE

ERD e ERS — Definizioni Tecniche ■

- **ERD (Event-Related Desynchronization):** Diminuzione dell'attività oscillatoria. Il cervello è attivamente impegnato. Valore positivo.
- **ERS (Event-Related Synchronization):** Aumento dell'attività oscillatoria. Il cervello è a riposo. Valore negativo.

$ERD = (Ampiezza_Rest - Ampiezza_Programming) / Ampiezza_Programming \times 100\%$

$ERD > 0 \rightarrow$ Desincronizzazione $\rightarrow$ Il cervello sta lavorando (problem solving)

$ERD < 0$ (ERS) $\rightarrow$ Sincronizzazione $\rightarrow$ Il cervello è rilassato

■ CONCETTI CHIAVE

Pipeline MNE-Python per l'analisi EEG:

- 1. `read_raw_edf` — Lettura del file EEG grezzo
- 2. `notch_filter(50 Hz)` — Rimozione rumore cavi elettrici europei
- 3. Calcolo IAF — Calibrazione individuale:

$IAF = 11.95 - (0.053 \times Et\grave{a})$ [l'Alpha decresce con l'età]

■ PIPELINE (continuazione)

- 4. Bandpass filtering — Isolamento della banda di interesse
- 5. Squaring dei campioni — Calcolo della potenza
- 6. Averaging sulle trial — Media delle ripetizioni
- 7. Calcolo ERD/ERS — Applicazione della formula

■ DOMANDE POSSIBILI

- "Cos'è l'ERD e come si calcola? Cosa ci dice sull'attività del programmatore?"
- "Perché l'EEG è preferibile all'fMRI per studiare i programmatori?"
- "Qual è il ruolo delle onde Alpha nel monitoraggio del carico cognitivo?"
- "Cosa è lo IAF e perché va calibrato individualmente?"

Blockchain, Metodologie Agili e Change Management

Capitolo 11 — Blockchain e Decentralizzazione

■ KEYWORDS

blockchain · distributed ledger · hash · nonce · consensus · PoW · PoS · smart contract · single point of failure · retroazione negativa · immutabilità

■ CONCETTI CHIAVE

Nel contesto del corso, la Blockchain è un **sistema decentralizzato a retroazione negativa** che tenta di costruire un regime PDS in un ambiente caotico, eliminando il "single point of failure".

```
blocco = {
  'indice': 1,
  'timestamp': '2026-03-28 10:00:00',
  'dati': 'Esecuzione Smart Contract',
  'hash_precedente': '0000abc123...', # crea la catena
  'nonce': 45632, # trovato con mining (PoW)
  'hash_attuale': '0000def456...'
}
```

Aspetto	Sistema Centralizzato	Blockchain
Single Point of Failure	■ Esiste	■ Eliminato (nodi distribuiti)
Trust	Basato su autorità centrale	Basato su protocollo matematico
Modificabilità	Alta (admin può cambiare)	Quasi nulla (immutabilità)
Regime di Mensch	PDS (dipende da chi gestisce)	Tende a PDS tramite consensus

■ COLLEGAMENTO FONDAMENTALE

AI + Blockchain: l'AI (Modeling) può analizzare gli smart contract per individuare vulnerabilità *prima* del deploy, spostando l'incertezza da SR/DCS a StS (prevedibile statisticamente).

Capitolo 12 — Lean, Agile ed Extreme Programming

■ KEYWORDS

7 sprechi Lean (Muda) · Manifesto Agile · XP Extreme Programming · TDD · pair programming · refactoring · CI/CD · cliente sul posto

■ CONCETTI CHIAVE

I 7 Sprechi del Lean Software Development (Muda) ■

#	Spreco	Esempio nel SW
1	Lavoro Parziale	Codice scritto ma non testato/integrato. Costo sospeso, non asset.
2	Processi Extra	Documentazione inutile; meeting non necessari; approvazioni a catena.
3	Funzionalità Extra (Overproduction)	Il 64% delle feature non viene mai usato.
4	Task Switching	Perdita di contesto cognitivo. Il "warm-up" è reale e misurabile via EEG.
5	Attese	Colli di bottiglia: PR non revisionate, test non eseguiti.
6	Movimento	Passaggi informativi inefficienti; re-spiegare il contesto.
7	Difetti	Bug scoperti tardi costano esponenzialmente di più.

Pratica XP	Fondamento Cognitivo	Beneficio
Pair Programming	Vygotsky ZPD + Risultato Comune (Malone)	Sistema riflessivo sempre attivo; trasferimento conoscenza
TDD	Feedback loop immediato	Riduce difetti; forza a pensare all'interfaccia prima
Refactoring continuo	Elimina spreco #2 e debito tecnico	Il sistema rimane modificabile nel tempo
CI/CD	Riduce Lavoro Parziale (spreco #1)	Ogni commit è potenzialmente deployabile
Cliente sul posto	Riduce Wicked Problem via feedback continuo	Requisiti emergenti gestiti senza crisi

■ COLLEGAMENTO FONDAMENTALE

Lean/Agile/XP sono la risposta pratica ai problemi Wicked in regime DCS/StS. La Matrice di Harris spiega perché funzionano: usano controllo di tipo "Clan/Group" invece di Outcome/Behavior quando il processo è troppo incerto.

Capitolo 13 — Change Management nel Software

■ KEYWORDS

corrective maintenance · perfective maintenance · adaptive maintenance · preventive maintenance · refactoring · domain analysis · variation points · Lampedusa

Strategia	Approccio
Prevenire il cambiamento	Requisiti dettagliati e stabili. Funziona solo per problemi Tame.
Anticipare il cambiamento	Domain analysis, variation points, architetture flessibili (plugin, DI).
Gestire il cambiamento	Manutenzione sistematica: correttiva, perfettiva, adattiva, preventiva.

Tipo Manutenzione	Descrizione
Correttiva	Fix di bug e difetti noti.
Perfettiva	Miglioramento performance o qualità interna (include refactoring).

Adattiva	Adattamento a nuovi ambienti: nuovo OS, nuova legge, nuova libreria.
Preventiva	Migliorare la manutenibilità futura prima che sorga un problema.

■ PARADOSSO DI LAMPEDUSA

Paradosso di Lampedusa (citato nelle slide): "Se vogliamo che tutto rimanga come è, bisogna che tutto cambi." (Il Gattopardo). Nel software: per mantenere il sistema funzionale bisogna continuamente cambiarlo. L'immobilità porta al decadimento.

■■ Il docente usa questo esempio per mostrare che "change" è spesso usato come buzzword per giustificare azioni non etiche. Risposta attesa: pensiero critico, non solo definizioni tecniche.

■ Ottimizzazione Esame — Piano 3 Giorni + Simulazione

■ Piano di Studio — 3 Giorni

■ GIORNO 1 (Domenica) — COMPRENSIONE

Obiettivo: Costruire la mappa mentale del corso. Non memorizzare, capire le relazioni.

- **Mattina (4h):** Cap 1-2 (natura software, Brooks, Ariane 5); schematizza i 4 regimi di Mensch su carta; Tame/Wicked con 3 esempi tuoi
- **Pomeriggio (4h):** Modello Impulsivo/Riflessivo (cuore di T05); 4 modelli cognitivi; Bateson (informazione, abduzione, double bind)
- **Sera (2h):** Tabella onde cerebrali; ERD/ERS e formula; Pipeline MNE-Python (7 passi)

■ *Fine giornata: scrivi 3 pagine a mano dei concetti non chiari. Il docente usa carta e penna — allenati.*

■ GIORNO 2 (Lunedì) — CONSOLIDAMENTO

Obiettivo: Approfondire i temi e costruire i collegamenti tra argomenti.

- **Mattina (4h):** Blockchain (struttura blocco, hash, nonce, consensus); 7 Sprechi Lean con esempi concreti; XP e fondamento cognitivo; 4 tipi di manutenzione
- **Pomeriggio (4h):** Disegna la mappa concettuale completa; pratica "Spiega Pair Programming attraverso Vygotsky, Malone e il modello impulsivo/riflessivo"
- **Sera (2h):** Matrice di Harris — prendi 5 contesti reali e classifica il tipo di controllo adatto

■ GIORNO 3 (Martedì) — RIPASSO ATTIVO

Obiettivo: Simulare l'esame orale. Rispondere ad alta voce.

- **Mattina (3h):** Le 10 domande di questa guida — rispondi ad alta voce, max 3-4 minuti a risposta
- **Mezzogiorno (2h):** Ripassa le keyword di ogni capitolo
- **Pomeriggio (2h):** Ripassa i punti deboli emersi dal ripasso mattutino
- **Sera:** Riposo. Leggi qualcosa di leggero. Dormi bene.

■ *Consiglio: il prof. Succi apprezza studenti che ragionano. Aggiungi sempre un collegamento interdisciplinare nelle risposte. Mostra di aver capito il filo rosso del corso.*

■ 10 Domande Reali con Risposte Modello

1. Cos'è il software? Quali sono le sue caratteristiche economiche peculiari?

Il software è il risultato di un atto creativo della mente umana. Ha una funzione di costo a "L": alto costo fisso di sviluppo, ma costo marginale di replica praticamente nullo. Questo abilita strategie di segmentazione e bundling. Non viene venduto ma licenziato (diritti morali non alienabili, economici alienabili; protezione 50 anni dalla morte). Le economie di scala del Fordismo esistono, ma la rigidità del modello fallisce di fronte all'incertezza intrinseca del software.

2. Quali sono le 4 difficoltà strutturali di Brooks? Collegli una al caso Ariane 5.

Brooks identifica: (1) Complessità — più stati possibili di qualsiasi sistema fisico; (2) Conformità — deve adattarsi a leggi e interfacce irrazionali preesistenti; (3) Modificabilità — il successo genera richieste di cambiamento continue; (4) Invisibilità — immateriale, difficile da visualizzare. Ariane 5 (1996) illustra Conformità e Complessità: un modulo di Ariane 4 riutilizzato senza verificare la compatibilità di contesto causò un overflow (64-bit in 16-bit), distruggendo il razzo (~8 miliardi di dollari di danni).

3. Descriva i 4 regimi di Mensch. Come si collegano all'approccio Agile?

I 4 regimi: PDS (Pseudo-Deterministico) — stabile "per ingegneria" ma con fragilità nascoste (illusione del tacchino); DCS (Caotico Deterministico) — equazioni precise ma sensibilità estrema alle condizioni iniziali (effetto farfalla); StS (Stocastico) — previsione statistica possibile; SR (Randomicità Stocastica) — caos puro, solo contenimento danni. Il Taylorismo/Waterfall assume software in regime PDS. Il software è quasi sempre DCS o StS. L'Agile risponde correttamente: usa cicli brevi (sprint) per gestire un sistema StS, stimando probabilisticamente invece di pianificare rigidamente.

4. Spieghi il modello Impulsivo/Riflessivo. Come si applica alle code review?

Strack & Deutsch (2004): il comportamento umano è risultato di due sistemi paralleli. Sistema Impulsivo: implicito, veloce, basato su memoria associativa, nessuno sforzo. Sistema Riflessivo: esplicito, lento, basato su ragionamento logico, alto sforzo cognitivo. Nelle code review, il sistema impulsivo tende a dominare ("sembra ok") sotto pressione temporale. Per attivare il sistema riflessivo servono protocolli: checklist esplicite, tempo adeguato, separazione tra lettura e valutazione. Gli strumenti AI di code review replicano il sistema impulsivo, non quello riflessivo.

5. Cos'è un problema Wicked? Perché il software è quasi sempre Wicked?

Un problema Wicked (Rittel & Webber) non ha formulazione definitiva (cambia mentre si risolve); le soluzioni non sono vere/false ma accettabili/non accettabili; è unico. Il software è Wicked perché i requisiti emergono durante lo sviluppo; il cliente non sa cosa vuole finché non vede un prototipo; ogni implementazione rivela nuove esigenze. Questo giustifica teoricamente l'Agile: il Waterfall (Tame) fallisce su problemi Wicked perché assume requisiti stabili e completi fin dall'inizio.

6. Descriva la Matrice di Harris e il suo uso nello sviluppo software.

La Matrice di Harris offre tre modalità di controllo: Outcome (misuro il risultato — utile quando conosco la relazione causa-effetto); Behavior (misuro i passaggi — utile quando conosco il processo alla perfezione, Taylorismo); Clan/Group (nessuna misurazione diretta — serve fiducia e team affiatato, Agile/Wicked). La scelta dipende dalla natura del problema: Tame/PDS → Behavior; Wicked/StS → Clan/Group. Errore comune: applicare Behavior (metriche rigide) a processi creativi, generando il fenomeno di Goodhart: "When a measure becomes a target, it ceases to be a good measure."

7. Cos'è l'ERD nell'EEG? Cosa ci dice sull'attività cognitiva di un programmatore?

ERD (Event-Related Desynchronization): diminuzione dell'attività oscillatoria cerebrale rilevabile via EEG. Formula: $ERD = (Ampiezza_Rest - Ampiezza_Task) / Ampiezza_Task \times 100\%$. Valore positivo = desincronizzazione = cervello attivamente impegnato. ERS (sincronizzazione) = riposo. Applicato ai programmatori: ERD in banda Alpha diminuisce durante debugging intenso o comprensione di codice complesso. Permette di misurare oggettivamente il carico cognitivo. L'EEG è preferito per portabilità, basso costo e alta risoluzione temporale rispetto all'fMRI.

8. Come si integra la Blockchain con i principi di Ingegneria del Software del corso?

La Blockchain è un sistema decentralizzato a retroazione negativa che tenta di costruire un regime PDS in un ambiente caotico, eliminando il "single point of failure". Il consensus protocol è il meccanismo di retroazione. Gli smart contract sono però problemi Wicked: i requisiti emergono dopo il deploy, ma il codice è immutabile — in tensione con i principi Agili. L'AI (Modeling) può analizzare gli smart contract prima del deploy per individuare vulnerabilità, spostando l'incertezza da SR/DCS a StS. Questo mostra l'integrazione di AI, Blockchain e metodologie Agili.

9. Cosa significa la "mente estesa" di Clark & Chalmers? Quale implicazione ha per il SW?

Clark & Chalmers: i processi cognitivi non si fermano ai confini del cranio. Otto (Alzheimer) usa un taccuino come memoria — funzionalmente equivalente alla memoria biologica. Per lo sviluppo software: IDE, documentazione, Stack Overflow, version control sono estensioni cognitive del programmatore. Un ambiente di sviluppo povero è una menomazione cognitiva reale. Gli LLM/Copilot sono la più recente espansione della mente estesa. Si collega al Pair Programming: il partner è un'estensione cognitiva che abbassa la ZPD di entrambi (Vygotsky).

10. Quali sono i 7 sprechi Lean? Colleghi almeno uno alla teoria cognitiva.

I 7 Muda: (1) Lavoro Parziale — codice non integrato; (2) Processi Extra — burocrazia; (3) Funzionalità Extra — 64% delle feature non usato; (4) Task Switching — perdita contesto; (5) Attese — colli di bottiglia; (6) Movimento — passaggi inefficienti; (7) Difetti — bug scoperti tardi costano esponenzialmente di più. Il Task Switching (#4) ha fondamento cognitivo preciso: la Mente Estesa mostra che il contesto cognitivo (IDE, artefatti mentali) è costruito su un problema specifico. Cambiare task richiede demolire e ricostruire tutto. Il "warm-up" cognitivo è reale e misurabile via EEG (picco di ERD all'inizio di ogni nuovo task). La pratica XP di lavorare su un solo task ha dunque fondamento neuroscientifico.

■ ■ Mappa Concettuale del Corso — Il Filo Rosso

```
SOFTWARE (atto creativo, immateriale, Wicked)
|
+-- Complessità strutturale (Brooks)
|   --> Sistema in regime DCS/StS (Mensch)
|       --> NON si può pianificare con Waterfall/Taylorismo
|       --> Serve Agile / XP / Lean
|
+-- Prodotto da PERSONE --> ha una dimensione COGNITIVA
|   |
|   +-- Come percepiamo? (Bateson: informazione come differenza)
|       |
|       --> Abduzione = base dell'AI
|   |
```

```

|      +-- Come usiamo la conoscenza?
|
|      |      Naive Scientist -> Cognitive Miser -> Motivated Tactician
|
|      |      --> Impulsive/Reflective Model (Strack & Deutsch)
|
|      |      --> Applicazione: Code Review, Debugging
|
|      |
|
|      +-- La mente e' ESTESA (Clark & Chalmers)
|
|      --> IDE, docs, strumenti = estensioni cognitive
|
|      --> Pair Programming (Vygotsky ZPD)
|
|      --> LLM/Copilot = nuove estensioni cognitive
|
|
+-- COME SI MISURA la cognizione?
|
|      --> EEG -> Onde cerebrali -> ERD/ERS -> MNE-Python
|
|      --> Predire qualita' SW dai pattern cognitivi
|
|
+-- COME SI CONTROLLA il processo?
|
|      --> Matrice di Harris (Outcome / Behavior / Clan)
|
|      --> Tame -> Behavior; Wicked -> Clan/Group
|
|
+-- TECNOLOGIE per SW distribuito e affidabile
|
|      +-- BLOCKCHAIN (sistema PDS decentralizzato)
|
|      |      --> Elimina Single Point of Failure
|
|      |      --> AI analizza Smart Contract -> sposta DCS->StS
|
|      +-- AI nello sviluppo
|
|          +-- Modeling (EEG, predizione qualita')
|
|          +-- Emulating (ML, LLM, test generation)

```

■ Glossario Rapido

Termine	Definizione in 1 frase
Abduzione	Ragionamento per pattern: trovare la spiegazione migliore per un insieme di osservazioni.
Cognitive Miser	Gli umani usano euristiche per minimizzare lo sforzo, accettando soluzioni "abbastanza buone".
Double Bind	Due messaggi contraddittori a livelli logici diversi creano un loop impossibile.
ERD	Diminuzione dell'attività oscillatoria EEG; indica sforzo cognitivo attivo.
Homeostasis	Tendenza di un sistema (team) a resistere al cambiamento per mantenere l'equilibrio.
IAF	Individual Alpha Frequency: frequenza alpha calibrata individualmente. $IAF = 11.95 - (0.053 \times Et\grave{a})$.
Muda	I 7 sprechi del Lean: lavoro parziale, processi extra, overproduction, task switching, attese, movimento, difetti.
PDS	Pseudo-Deterministico: sistema che appare stabile ma nasconde fragilità (illusione del tacchino).
Schismogenesi	Processo di divergenza escalante tra due entità (complementare o simmetrica). Concetto di Bateson.

Single Point of Failure	Componente la cui rottura provoca il fallimento dell'intero sistema — eliminato dalla blockchain.
Wicked Problem	Problema senza formulazione definitiva; soluzioni non vere/false ma accettabili; cambia mentre si risolve.
ZPD	Zona di Sviluppo Prossimale (Vygotsky): spazio tra ciò che so fare da solo e con un mentore.